CHAOS OS

How to pitch it, end to end (STAR format)

For presentations, vivas and placement interviews. Every number below was measured in the lab.

Repository: https://github.com/Prarthan-B-P/chaos-os

1. The pitch at three lengths

10 SECONDS

"CHAOS OS breaks a live web shop on purpose, lets AI and classic responders fix it behind a safety shield, measures every recovery to the second, and stops releases that cannot recover."

30 SECONDS

"Every app eventually breaks: a cache crashes, a database slows down. Teams usually guess the fix, mostly 'restart it', and our data shows that guess is wrong for most failures. CHAOS OS breaks a live shop on purpose, lets five kinds of responders fix it, from a classic runbook to machine learning to an AI agent, and measures every recovery to the second. A safety shield blocks dangerous actions and a human approves risky ones. An always-on AI on-call engineer heals failures nobody announced in about 25 seconds, and Jenkins refuses to ship any release that cannot recover."

2 MINUTES (ADD AFTER THE 30 SECONDS)

"We found three things. First, there is no universal fix: each failure has its own best action, and a wrong one costs 450 to 1,100 failed requests instead of 30 to 250. Second, a small learning algorithm, a contextual bandit, learned the right fixes from our experiment logs without anyone writing rules, and even handled a failure it had never seen, where the classic runbook failed every time. Third, we tested gpt-4o-mini carefully: one paragraph of advice changed its behaviour more than giving it tools did, in both directions, and the best setup combined both.

Then we made it run like production: users are simulated, Prometheus watches an SLO, Alertmanager pages our on-call agent, and in 21 surprise failures it healed 18 by itself; the other 3 had no safe fix and it handed them to a human. Every release goes through Jenkins with security scanning, real failure tests and a surprise on-call drill, and it all deploys with HTTPS and a login in one command."

2. The real-world problem

Outages are long and expensive because the fix is a guess. When something breaks at 3 a.m., an engineer must work out what failed and which fix applies. The reflex "restart it" is often wrong: for a slow network to the cache, restarting the cache does nothing. Every minute of guessing is lost revenue and unhappy users. The industry measures this as **MTTR** (mean time to recover).

What CHAOS OS changes: measure which fix works for which failure, learn it automatically, act in seconds behind a safety shield with a human for risky actions, and block releases that cannot recover before customers ever see the outage.

3. The whole project as one STAR story

The project

STAR	What happened
Situation	Production systems fail constantly. Teams rely on untested runbooks, automation can make incidents worse, and CI pipelines only test code when everything is healthy, never recovery.
Task	Build a lab that proves, with measurements, which recovery works for which failure; makes automated recovery safe; runs like a real production service; and turns resilience into a release gate.
Action	Built an 11-service lab (FastAPI shop, Redis, Postgres, Toxiproxy fault injection, Prometheus, Alertmanager, Grafana, synthetic users, on-call agent, React dashboard) with Docker Compose; recovery actions as Ansible playbooks; a safety validator; five responders (runbook, expert rules, LinUCB bandit, gpt-4o-mini, LLM agent with tools); SLO burn-rate alerting; and a 10-stage Jenkins pipeline with security, resilience and on-call gates.
Result	More than 200 measured trials. Bandit 88% best-action accuracy vs 33% for the runbook, and it generalised to an unseen failure (3/3 in 0.8 s, runbook 0/3). On-call agent healed 18 of 21 surprise failures (median 25.8 s to healthy traffic). Zero unsafe actions. Jenkins blocks a one-character bug that unit and smoke tests miss.

4. STAR stories for each part

Use one of these when someone asks "tell me about a part you built", or for a specific interview question.

Measurement: "How do you know it recovered?"

STAR	What happened
Situation	Prometheus scrapes every 5 seconds, too coarse to measure recoveries of 1-2 seconds.
Task	Define recovery precisely and measure it fairly for every responder.
Action	Wrote an open-model load generator that times every request (20 req/s) and a unit-tested definition: 1-second windows, healthy at 99% success and p95 at most 300 ms, recovered after 5 healthy windows.
Result	Recovery is measured below one second; for example cache bypass heals a dead Redis in 1.8 s, a restart of the API takes 14.6 s.

Safety: "What stops the AI from doing damage?"

STAR	What happened
Situation	Automated remediation can cause outages (restart loops, acting on the wrong service), and LLMs can propose anything.
Task	Make every action, from any responder, provably safe.
Action	Built a safety validator from a YAML policy (approved environment, allowlist, no repeats, max 3 actions, approval for high impact), a second allowlist in the Ansible playbooks, human approval in the dashboard, and a red-team policy that proposes destructive actions on purpose.
Result	Zero unsafe actions executed. `drop_database` and `restart_grafana` were blocked; the validator even stopped the LLM agent from repeating a database restart.

Machine learning: the bandit

STAR	What happened
Situation	Hand-written expert rules work but need a person to study every new failure.
Task	Learn the right fix from experiment logs alone, using only observable symptoms.
Action	Implemented LinUCB with 8 symptom features and a reward of heal speed minus safety cost, trained only on training failures, and tested it on held-out failures after freezing it.
Result	88% best-action accuracy (expert rules 84%, runbook 33%); recovered an unseen Redis network partition 3/3 in 0.8 s where the runbook failed 0/3.

Generative AI: the LLM ablation

STAR	What happened
Situation	LLM agents for operations are hyped but rarely tested on real failures.
Task	Find out what actually makes gpt-4o-mini a better responder: tools or advice.
Action	Built a tool-using agent (PromQL, logs, container states, readiness check, must investigate before deciding) and ran a 2 x 2 ablation (tools x generic SRE advice) over 56 real trials.
Result	Advice changed the habit, not the reasoning: Redis latency 50.1 s to 3.3 s, but it then never restarted a crashed database. Tools alone added 8-12 s. Tools plus advice was the most robust (10/14). Honest lesson: test prompts like code.

SRE: the on-call agent

STAR	What happened
Situation	In real life nobody tells you what broke; alerts must be meaningful and responders must not act on stale alerts.
Task	Heal failures nobody announced, safely, like a human on-call engineer.
Action	Synthetic users measure the SLI; Prometheus pages on a multi-window burn rate (14.4x, Google SRE Workbook, unit-tested); Alertmanager wakes the agent; it confirms with its own traffic, fixes through the safety validator, waits for human approval on risky actions (no answer = no action), escalates instead of looping, and writes a post-mortem.
Result	18 of 21 surprise failures healed; median 11.8 s to the alert and 25.8 s to healthy traffic; the 3 unfixable ones were escalated correctly; 8 risky actions ran only after approval.

DevOps: the pipeline and security

STAR	What happened
Situation	Releases were tested only for correctness, and the first security scan found real issues.
Task	Make resilience, alerting, security and on-call behaviour release gates.
Action	Jenkins (multibranch, job as code, isolated lab per build): build, 45 unit tests, alert-rule tests, gitleaks + Trivy + SBOMs, deploy, smoke, 6-fault resilience gate, surprise on-call drill. Fixed every finding: upgraded ansible-core and Vite, 7 containers made non-root, newer base images (61 CVEs to 0 in the frontend image).
Result	Main passes all stages in under 6 minutes; the demo branch with a one-character bug passes unit and smoke tests but is blocked by the resilience gate.

5. The live demo flow (7 minutes)

Time	Show	Say
0:00	Dashboard Overview	Everything here is measured.
0:40	On-call → Break it (Surprise me)	I just broke the shop; the agent was not told.
1:00	Incident timeline	Users noticed, the SLO burned, the agent was paged, confirmed, fixed it safely.
2:00	Post-mortem	gpt-4o-mini wrote it from the incident record only.
2:45	Break it: postgres_down → Approve	Risky action, so it asks me; no answer means no action.
3:45	Incidents → LLM agent trial → tool trail	It investigated with real tools before deciding.
4:30	Grafana SLO row	The burn rate crosses the 14.4x paging line.
5:15	Jenkins main (green) vs demo branch (red)	A one-character bug passed tests; the resilience gate blocked it.
6:15	Ask the lab	Ask it anything; the answer comes from our data.

Start with start.bat (local) or share.bat (public link with login) 10 minutes before.

6. Numbers to remember

Fact	Number
Services in the lab	11 containers
Failures / actions / responders	7 / 6 / 5 (+2 ablation variants, +1 red team)
Wrong fix vs right fix	447-1,128 vs 33-252 failed requests per incident
Bandit vs expert vs runbook	88% / 84% / 33% best-action accuracy
Unseen Redis partition	bandit 3/3 in 0.8 s; runbook 0/3
LLM ablation	56 trials; advice: 50.1 s → 3.3 s on Redis latency; tools +8-12 s
On-call campaign	18/21 healed; 11.8 s to alert; 25.8 s to healthy
CI	10 stages, under 6 min; 45 unit + 4 alert-rule tests
Security	0 secrets; 61 → 0 CVEs in the frontend image; 7 containers non-root
Unsafe actions executed	0

7. Five tough questions, short answers

ISN'T THIS JUST RESTARTING CONTAINERS?

No. Restarting is the wrong fix for most failures in our data. The project is about choosing the right fix, proving it with measurements, and doing it safely.

WHY NOT USE AN EXISTING CHAOS TOOL?

Chaos Mesh or Gremlin inject faults. We compare recovery strategies, learn from them, and gate releases on recovery.

CAN THE AI BREAK PRODUCTION?

It only proposes. A safety validator and an Ansible allowlist check every action, and risky ones need a human. Zero unsafe actions ran.

WHAT ARE THE LIMITATIONS?

One laptop, 2-3 trials per test, and one compound failure nobody solved. Next steps: multi-step planning, a Kubernetes port, more trials with confidence intervals.

WHAT DID YOU PERSONALLY LEARN?

Measure before you automate, put safety layers around any automation, and test AI prompts like code, because one paragraph changed recovery times by 15x in both directions.